USER DOCUMENTATION

Brusselator Reaction-Diffusion Solver

Final Project

A practical guide to setting up the environment, building the solver,
running simulations, performing parameter studies, testing and visualising results.

Document purpose and how to use this guide

This document is written for someone who has not previously used this project. It explains the steps required to obtain the project files, prepare the Docker environment, compile the C++ programs, run the Brusselator solver, execute the tests, inspect the notebooks and visualise the generated results.

Follow the sections in order the first time you use the project. Commands are shown in shaded monospaced blocks. Unless a step explicitly says otherwise, commands in Sections 3–10 are executed in a terminal inside the Docker container.

Command-line conventions

- Host terminal: the terminal on your own computer, outside the Docker container.
- Container terminal: the terminal after startContainer.sh has launched the container.
- Project root: /workspace inside the container.
- Build directory: /workspace/build, where the compiled executables are created.

The startup script supplied with this project is configured for a Docker environment and includes WSLg/X11 display forwarding. On Windows, use a WSL2 distribution with Docker Desktop and WSLg configured if you want to launch graphical applications such as ParaView from the container. The solver and command-line tests can still be run without opening a graphical application.

1. About the project

The project solves a two-dimensional, time-dependent Brusselator reaction-diffusion system. The two concentration fields, C1 and C2, evolve through diffusion and nonlinear chemical reaction terms. Spatial derivatives are discretised using finite differences, while different numerical schemes are used to advance the solution in time.

The implemented time-integration methods are:

- Explicit Euler
- Classical fourth-order Runge-Kutta (RK4)
- Implicit Backward Euler, solved using Newton iterations

The solver writes simulation data in VTK ImageData (.vti) format. These files can be inspected with ParaView. The project also contains C++ unit tests, Python component tests, parameter-study scripts and a validation notebook that compares results with reference data.

2. Project files and directory layout

The main directories and files are organised as follows:

```
scientific_computing_project/
├── CMakeLists.txt
├── README.md
├── requirements.txt
├── startContainer.sh
├── build/
├── configs/
│   ├── Dockerfile
│   ├── custom_bashrc
│   ├── jupyter_lab_config.py
│   ├── neovim_config
```

```
│   └── python.md
├── include/
│   ├── brusselator.hpp
│   ├── integrator.hpp
│   ├── output.hpp
│   └── simconfig.hpp
├── src/
│   ├── brusselator.cpp
│   ├── integrator.cpp
│   ├── output.cpp
│   └── solver.cpp
├── resources/
│   └── final_output_30s.vts
├── scripts/
│   ├── varyCb.sh
│   └── varyDt.sh
├── tests/
│   ├── unit/
│   └── component/
├── validation/
│   └── validation.ipynb
└── results/
```

Location	Purpose
configs/	Dockerfile and environment configuration files.
include/ and src/	C++ header and implementation files.
build/	CMake-generated build files and compiled executables.
resources/	Reference solution used for numerical validation.
scripts/	Bash scripts for automated parameter variation.
tests/unit/	C++ unit tests for individual numerical components.
tests/component/	Python component test and parameter-study notebooks.
validation/	Notebook for comparing computed and reference solutions.
results/	Generated simulation output and parameter-study results.

3. Prerequisites

Before starting, make sure the following are available on the host computer:

- Docker Engine or Docker Desktop.
- A terminal that can run Bash shell scripts.
- The complete project directory, including the Dockerfile and source files.
- For graphical applications from a Windows host: WSL2/WSLg and the display forwarding setup expected by startContainer.sh.

The Docker image contains the project build environment. You do not need to install the C++ compiler and project libraries individually on the host if you build and run the project inside the supplied container.

4. Build the Docker image

Open a host terminal and change directory to the project root—the directory containing CMakeLists.txt, startContainer.sh and the configs folder.

```
cd /path/to/scientific_computing_project
```

Build the Docker image using the Dockerfile inside configs/:

```
docker build -t brusselator-image -f configs/Dockerfile .
```

The command creates a local Docker image called brusselator-image. The final dot is important: it specifies the project root as the Docker build context.

5. Start the Docker container

Still in the project root on the host, make the startup script executable:

```
chmod +x startContainer.sh
```

Start the container:

```
./startContainer.sh
```

The script starts a container from brusselator-image and mounts the current project directory to /workspace inside the container. This means files created under /workspace/results in the container are also visible in the project directory on the host.

The container hostname is configured as mycfdsolver-container. Verify the container environment:

```
hostname
pwd
ls
```

The hostname command should report mycfdsolver-container. The project should be available at /workspace.

Important: launch startContainer.sh from the project root. The script uses $(pwd) for the host-side directory mount, so starting it from another directory may mount the wrong location.

6. Compile the solver and unit-test executable

Inside the container, navigate to the project root:

```
cd /workspace
```

Create and enter the build directory:

```
mkdir -p build
cd build
```

Configure the project with CMake:

```
cmake ..
```

Compile the configured targets:

```
make -j$(nproc)
```

CMake reads the root CMakeLists.txt and generates the build system. GNU Make then compiles the configured targets. The parallel-build option -j$(nproc) allows Make to use the available CPU cores. The build is configured to produce the main brusselator executable and the unit_tests executable.

Check that the executables are present:

```
ls -l brusselator unit_tests
```

If either executable is missing, inspect the CMake output for configuration or compilation errors before proceeding.

7. Run a simulation

7.1 Command format

Run the solver from the build directory using the following argument order:

```
./brusselator <solver> <nx> <ny> <t_end> <dt> <write_interval> <Ca> <Cb>
```

Argument	Meaning
solver	Time-integration method: explicit, implicit or rk4.
nx	Number of grid points in the x-direction.
ny	Number of grid points in the y-direction.
t_end	End time of the simulation.
dt	Time-step size.
write_interval	Interval at which output is written.
Ca	First Brusselator reaction parameter.
Cb	Second Brusselator reaction parameter.

7.2 Example runs

Explicit Euler:

```
./brusselator explicit 40 40 30 0.01 1 1 3
```

Implicit Backward Euler:

```
./brusselator implicit 40 40 30 0.01 1 1 3
```

Classical RK4:

```
./brusselator rk4 40 40 30 0.01 1 1 3
```

These examples use a 40 × 40 grid, a final time of 30, a time step of 0.01, a write interval of 1, Ca = 1 and Cb = 3. The output is written by the solver into method-specific directories under results/.

8. Run the C++ unit tests

The unit tests check individual building blocks of the numerical implementation. The test source files are located in tests/unit/ and cover the Laplacian operator, reaction terms and the Jacobian used by the implicit solver.

From the build directory, execute:

```
cd /workspace/build
./unit_tests
```

The test executable is built together with the main solver by the CMake and Make commands in Section 6. There is no need to compile the test source files manually.

9. Set up Python and run component tests

Python is used for component-level testing and notebook-based analysis. The Python dependencies are listed in requirements.txt.

9.1 Create and activate a virtual environment

```
cd /workspace
python3 -m venv .venv
source .venv/bin/activate
```

When activation succeeds, the environment name (typically .venv) appears in the terminal prompt. Keep the environment activated while installing packages, running pytest and launching Jupyter.

9.2 Install the dependencies

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

The packages installed are those specified by the project's requirements.txt file.

9.3 Run the component test

The Python component test is located at tests/component/test_component.py. It exercises the solver as a component rather than testing an individual C++ function.

```
cd /workspace
source .venv/bin/activate
pytest tests/component/test_component.py
```

The compiled solver must already exist in the build directory. If pytest is not found, confirm that the virtual environment is active and that the dependencies were installed successfully.

10. Run parameter variation studies

The project contains two Bash scripts that automate repeated simulations and organise each run's output into a separate parameter-specific directory. Run these scripts from /workspace/scripts, because their relative paths refer to ../build and ../results.

```
cd /workspace/scripts
chmod +x varyCb.sh varyDt.sh
```

10.1 Vary Cb using RK4

varyCb.sh performs an RK4 study with a 40 × 40 grid, end time 30, time step 0.01, write interval 1 and Ca = 1. The supplied values are assigned to Cb.

```
./varyCb.sh Variation_Cb Cb 1.5 2.0 2.5 3.0
```

The script runs one simulation for each supplied Cb value and organises the output in directories such as results/Variation_Cb/Cb_1.5 and results/Variation_Cb/Cb_2.0.

10.2 Vary dt using Implicit Backward Euler

varyDt.sh performs an implicit-solver study with a 40 × 40 grid, end time 5.0, write interval 1, Ca = 1 and Cb = 3. The supplied values are assigned to dt.

```
./varyDt.sh Variation_dt dt 0.01 0.02 0.03 0.04 0.05
```

The script creates a separate directory for each time step and moves the VTK files, the generated start.sh and newton_history.txt into the corresponding directory.

10.3 How the scripts organise results

The solver creates method-specific output directories, for example Runge-Kutta/ and Implicit_Euler/. The Bash scripts create additional parameter-specific directories under Variation_Cb/ or Variation_dt/ and move the output files into them. Seeing both kinds of directories is expected: one level is used by the solver, and the other organises the parameter study.

To change the values being studied, supply a different list of values after the variation name and parameter name. The scripts use their own predefined settings for the other simulation parameters.

11. Open the Jupyter notebooks

The project includes one validation notebook and two component parameter-study notebooks. Start Jupyter from the directory containing the notebook you want to open.

11.1 Open the validation notebook

```
cd /workspace
source .venv/bin/activate
cd /workspace/validation
jupyter notebook
```

Jupyter prints a local URL in the terminal. In the configured Docker setup, open the notebook interface in your host browser at:

```
http://127.0.0.1:8888/tree
```

Select validation.ipynb from the file list.

11.2 Open the component parameter-study notebooks

```
cd /workspace
source .venv/bin/activate
cd /workspace/tests/component
jupyter notebook
```

Open either notebook from the browser interface:

- parameter_cb_rk4.ipynb — investigates Cb variation using the RK4 solver.
- parameter_dt_implicit.ipynb — investigates time-step variation using the implicit solver.

If the browser does not open automatically, copy the URL printed by Jupyter into the browser. The address http://127.0.0.1:8888/tree is the expected local interface for the supplied port mapping. If port 8888 is already in use, stop the other Jupyter server or use the alternative URL/port printed by the new server.

To stop a Jupyter server, return to the terminal where it is running and press Ctrl+C.

When finished with Python work, deactivate the virtual environment if desired:

```
deactivate
```

12. Visualise results with ParaView

Simulation output is written in VTK ImageData (.vti) format. ParaView can open these files to display concentration fields and examine their evolution.

From a container terminal, launch ParaView:

```
paraview
```

In the ParaView interface:

1. Choose File → Open.
2. Navigate to the relevant results directory and select a .vti file.
3. Click Apply in the Properties panel.
4. Choose the field of interest (for example, C1 or C2) in the Color By selector.
5. Use the available time controls if the data has been loaded as a time series.

ParaView is a separate graphical application; it is not part of the solver executable. The supplied startup script forwards display-related settings for graphical applications. If ParaView does not open, first confirm that the host's WSLg/X11 display integration is working.

13. Numerical validation

The reference solution supplied with the project is resources/final_output_30s.vts. The notebook validation/validation.ipynb is provided to compare simulation results against this reference data. Open the notebook using the procedure in Section 11.1 and execute its cells.

The validation notebook and reference file are part of the project. The exact comparison measures and interpretation should be taken from the notebook itself; this guide does not assume a particular error metric or numerical accuracy value.

14. Troubleshooting and common checks

Symptom	Checks / action
Docker build cannot find the Dockerfile	Run the build command from the project root and retain -f configs/Dockerfile .
Project files are not visible in the container	Check that startContainer.sh was launched from the project root; verify /workspace.
brusselator or unit_tests is missing	Return to /workspace/build, rerun cmake .. and make -j$(nproc), and inspect errors.
Permission denied when running a script	Use chmod +x script_name.sh, then run it with ./script_name.sh.
Parameter script cannot find the executable	Run the script from /workspace/scripts as described in Section 10.
pytest command is not found	Activate .venv and install dependencies with pip install -r requirements.txt.
Notebook page does not load	Check that Jupyter is still running, use the URL printed in its terminal, and confirm port 8888 is available.
ParaView does not launch	Check the host display/WSLg integration and the display forwarding configured by startContainer.sh.

15. Quick command reference

Task	Command
Build Docker image (host, project root)	docker build -t brusselator-image -f configs/Dockerfile .
Make startup script executable	chmod +x startContainer.sh
Start container (host, project root)	./startContainer.sh
Configure and build (container)	cd /workspace/build && cmake .. && make -j$(nproc)
Run Explicit Euler	./brusselator explicit 40 40 30 0.01 1 1 3
Run Implicit Backward Euler	./brusselator implicit 40 40 30 0.01 1 1 3
Run RK4	./brusselator rk4 40 40 30 0.01 1 1 3

Run unit tests	cd /workspace/build && ./unit_tests
Create and activate Python environment	cd /workspace && python3 -m venv .venv && source .venv/bin/activate
Install Python packages	pip install -r requirements.txt
Run component tests	pytest tests/component/test_component.py
Run Cb study	cd /workspace/scripts && ./varyCb.sh Variation_Cb Cb 1.5 2.0 2.5 3.0
Run dt study	cd /workspace/scripts && ./varyDt.sh Variation_dt dt 0.01 0.02 0.03 0.04 0.05
Open validation notebook	cd /workspace/validation && jupyter notebook
Open component notebooks	cd /workspace/tests/component && jupyter notebook
Launch ParaView	paraview

16. Notes and assumptions

This guide follows the project structure, commands and script behaviour documented in the project's README and the supplied final-project description. The parameter-study scripts use their configured default simulation settings; inspect the scripts before changing those settings. The permitted time-step range for the implicit solver is not specified in this guide, so use the limits defined for your project or implementation. The exact CMake test target and Python package list are determined by CMakeLists.txt and requirements.txt respectively.